POLITECNICO
MILANO 1863

Adaptative and Nonlinear Control
Individual Project

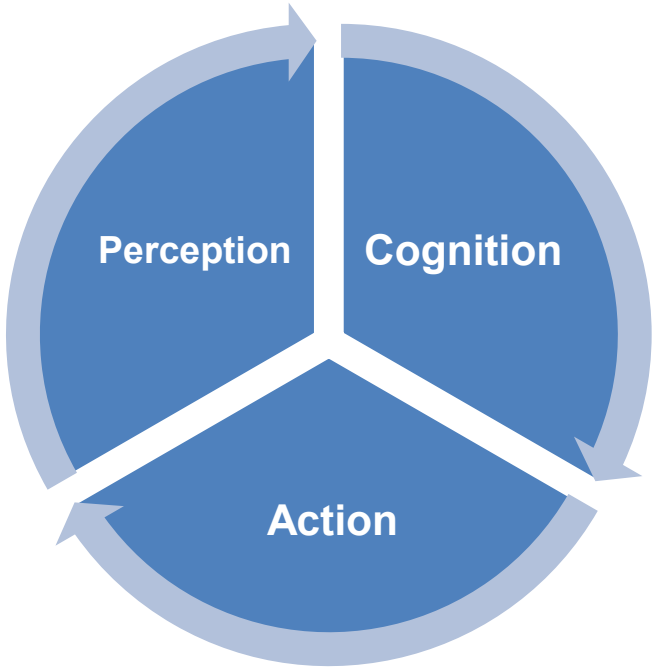
1. Introduction

1.1 Introduction

Path Planning – Context & Architecture integration

- **Autonomous:** the ability of a system to **achieve its goals** while **operating independently of external** controls, so itself is capable of these 3 fundamental actions:

1. **Perception**: extract information from the **sensors** (both exteroceptive and proprioceptive) and **update the map and location** of the system (performed by the SLAM).
2. **Cognition**: by considering its state, the goal and additional parameters (environmental variables, performance constrains, etc.), **obtaining a plan to follow** (path, orientation & trajectory).
3. **Action**: given the desired maneuver, the system **commands the inputs** to properly follow it.



In the previous project, the focus was made into the high-frequency action phase, while now the cognition process is the highlighted one. In particular, the **path planning** is going to be the main topic of the presentation.

1.1 Introduction
Path Planning

- The motion planning problem is about finding a sequence of valid configurations (a geometric path) that moves a robot from an initial start state x_0 to a specified goal state x_f. But this path needs to satisfy some fundamental conditions, listing:

    1. **Safety**: must be **collision-free**, both for expected and unexpected obstacles.

    2. **Feasible**: must satisfy the **physical constrains** of the UAV.

    3. **Completeness:** it is **guaranteed** to return a **feasible solution** if one exists, and crucially, it will return "**FAIL**" in a finite amount of time **if no such solution exists**.

    4. **Optimality:** it doesn't just find *any* path, but it finds the **path with** the absolute **minimum cost** among all possible paths from start to goal.

* **Efficiency**: for the path planning to not be the **bottleneck of UAV performance**, it needs to use the least amount possible of **computer resources** to work at desired frequency.

1.1 Introduction
Path Planning – Different Methods

- The first two algorithms developed for path planning are **Dijkstra's and A***. They will serve as a basis for explaining the behavior of more advanced ones, and the main topic of this presentation, D* lite and AI powered methods.

- Dijkstra's method:

Initialize node
Set starting node value to 0

→

Explore unvisited nodes
Calculation of new cost (initial node cost + edge distance)

→

Update
Check if this new cost is cheaper than the previous

→

Close
Once every neighbor has been checked, it is already known the shortest path to this node. Check as visited.

→

Repeat
Pick the next unvisited node

While this method is **complete and optimal**, there are two main disadvantages that makes it obsolete: need to **perfectly know the environment**; any modification in it make necessary the complete re-computation of the path, and because it performs the evaluation in the full grid, its **highly cost** in high order systems really limits its applications.

1.1 Introduction

Path Planning – Different Methods

- **A* method**: by using a **heuristic modification** to the evaluation function, the algorithm rewards the points that closes distance with the goal (with more or less intensity depending on the weighting of this function).

$$f = g + \omega h$$

With f being the total cost function of a node, g is Dijkstra's cost function, h is the heuristic function (contains, for example, the straight distance to the goal) and ω is the weighting parameter.

- Same problem with environment modification as Dijkstra, but **more computationally effective**.

- For an autonomous UAV to have a decent performance, it needs an algorithm that **resolves faster unexpected changes** in the environment. For urban zones specially, where numerous variables and events are common, but also for non previously explored regions. For this matter the **D*** and **D* lite** algorithm exist.

2. D* METHOD FOR PATH PLANNING

2.1 D* METHOD FOR PATH PLANNING OVERVIEW

- **Static Planners (A*/Dijkstra):** When an environment changes (e.g., a moving obstacle appears), static planners must discard the old path and **solve the entire search** task from scratch. This is extremely **inefficient** and computationally intractable in very large or dynamic environments.

- **D* (Dynamic A*):** instead of solving again the entire path, **D* reuses information** from previous searches, reducing significantly the number of operations needed for each iteration of path planning.

Therefore, when an unexpected obstacle is detected, it only updates the part of the **path** that is **affected by its presence**, reducing again the computations needed for solving the problem.

- **D* Lite:** while having the same navigational behavior as D*, its algorithm is simpler (based on another algorithm called Lifelong Planning A*) and optimized significantly, but sticking to the same principles as its base one [1].

2.1 D* METHOD FOR PATH PLANNING BEHAVIOR

- **D* lite** utilizes two new metrics for each cell, where *g* represents the cost from the previous search and *rhs* the current iteration one.

$$rhs = g \rightarrow Consistent \rightarrow \text{Save for next iteration}$$
$$rhs \neq \mathrm{g} \rightarrow Inconsistent \rightarrow \text{Update queue}$$

- When updating new values of the nodes, it checks if the addition of the obstacle is affecting at all that node. In the negative case, the algorithm **doesn't update it** and that's the limit point that remains the same as the previous iteration. This effects **limits the search** of the points **only to surroundings** of the **obstacle**, reducing computations as said.

- Another variable used to increase efficiency is $\boldsymbol{k_m}$, that reduces the **re-sorting** of the update queue that would be produced in each iteration because of new values of heuristic.

2.2 D* METHOD FOR PATH PLANNING
D* LITE RISK-AWARE

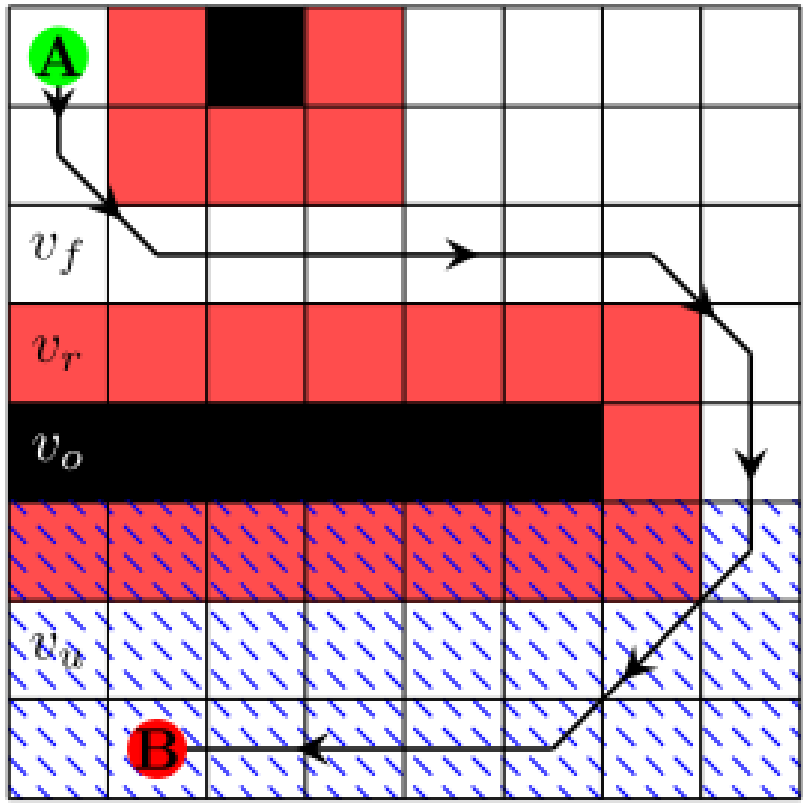
- **D* lite risk-aware** algorithm [2] is a modification of the D* lite that takes **risk** into account**.** This is introduced mathematically in the cost as:

$$\text{Cost} = \text{Distance (both real and heuristic)} + \lambda(\text{risk})$$

- This risk is increased in points **too close to obstacles** and unknown/**unexplored points**, contrary to D* lite algorithm, which is optimistic with respect to that points.

- Despite all this, due to possible appearance of sudden obstacles and reaction time of the algorithm not being suited for certain speeds, it is recommended [2] to incorporate in the UAV a **reactive system** to avoid collisions.

2.2 D* METHOD FOR PATH PLANNING
D* LITE RISK-AWARE

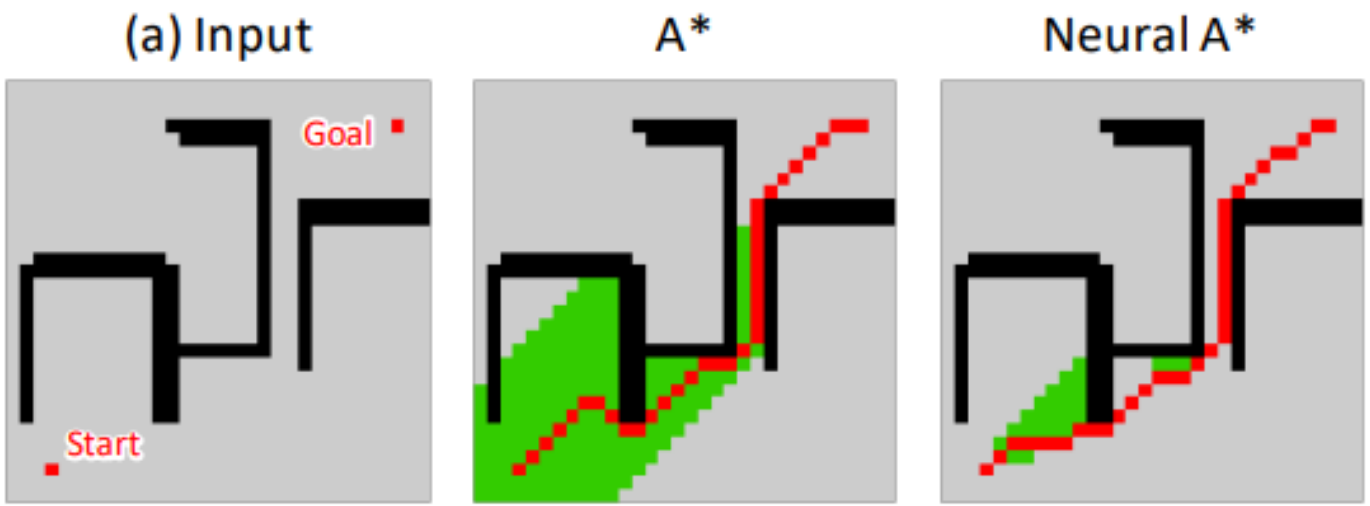


(a)

2.3 D* METHOD FOR PATH PLANNING
D* LITE WITH NEURAL HEURISTIC

- This **D* lite** modification changes the traditional heuristic of Euclidean distance from that node to the goal/UAV with **Guidance Map** generated by a Convolutional Neural Network (CNN).

- Drastically reduces **node exploration** (up to 90%), allowing the UAV to find optimal paths with significantly less CPU power [3].



3. Conclusions

3.1. Conclusions

- **Computational Efficiency via Incremental Planning**: D* Lite remains the industry standard for dynamic environments because it only updates the "inconsistent" nodes affected by new obstacles, avoiding recalculating the entire map.

- **Safety Through Risk-Awareness (D+*)**: Modern field-resilient navigation requires more than just avoiding obstacles; by treating unknown spaces and proximity to walls as "Risk Costs," we can ensure UAVs maintain a safety buffer and avoid the "optimistic" shortcuts that lead to dead ends.

- **Neural Guidance as the Future of Search**: The integration of Deep Learning (Neural A*/D*) allows search-based algorithms to include visual "intuition." This drastically reduces node exploration—often by over 80%—while guaranteeing optimality and success that are missing from pure AI models.

Sources

- [1] S. Koenig and M. Likhachev, "D* lite," in *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI)*, 2002, pp. 476-483.
- [2] S. Karlsson, A. Koval, C. Kanellakis, and G. Nikolakopoulos, "D+*: A risk aware platform agnostic heterogeneous path planner," Expert Systems with Applications, vol. 215, p. 119408, 2023.
- [3] R. Yonetani, T. Taniai, M. Barekatain, M. Nishimura, and A. Kanezaki, "Path planning using Neural A* search," arXiv preprint arXiv:2009.07476v3, Jul. 2021.

POLITECNICO
MILANO 1863

THANKS FOR YOUR ATTENTION